Why Vector DBs Alone Are Not Enough for Agents

Semantic similarity finds related chunks. Agents need relationships, exact answers, permissions, and structured reasoning that vectors cannot provide.

Rajesh Mane

Senior AI Engineer

What Vector Databases Do Well

🔍 Vectors excel at semantic similarity over unstructured text. Embed a query, find the closest chunks, and return context the LLM can reason over.

📚 They power RAG by retrieving relevant paragraphs from documents, wikis, and support tickets to ground LLM responses in real data.

🌐 They handle fuzzy queries where the user does not know exact keywords. Semantic search finds meaning even when phrasing differs from the source.

⚡ They scale to billions of embeddings with sub second latency. Purpose built stores like Qdrant, Pinecone, and pgvector handle production workloads reliably.

✅ **VECTORS ARE SOLVED, NOT SUFFICIENT**

Vector search is a solved problem. The question is no longer whether to use it but what to pair it with for the queries vectors cannot answer.

Three Gaps Vectors Cannot Fill

🔗 **NO RELATIONSHIPS BETWEEN ENTITIES**

Vectors store flat chunks with no connections. They cannot answer queries that require joining facts across entities because the relationship is not encoded in the embedding.

🎯 **NO EXACT OR STRUCTURED ANSWERS**

Similarity search returns the closest match, not the correct answer. Contract renewal dates, order counts, and account balances need precise values, not similar paragraphs.

🔒 **NO NATIVE ACCESS CONTROL**

Flat vector stores have no permission model. Every chunk is equally retrievable regardless of who asks. Enterprise deployment requires permission aware retrieval from the data layer.

Similarity Is Not Relevance

WHY CLOSEST DOES NOT MEAN CORRECT

1

Embedding distance does not equal answer quality

Vector search ranks by proximity in embedding space, not by whether the chunk actually answers the question. A semantically similar paragraph may be contextually irrelevant.

2

Reranking helps but adds cost and latency

Cross encoder rerankers rescore retrieved chunks by relevance, improving precision. But they add an extra inference call per query, increasing latency and compute cost.

3

Hybrid search consistently outperforms pure vectors

Combining dense vector similarity with sparse BM25 keyword matching catches exact terms, IDs, and proper nouns that embeddings compress away.

💡 WHAT EMBEDDINGS LOSE

Embedding models compress meaning into fixed dimensions. Nuance, negation, and specificity are the first casualties. The word 'not' changes everything but barely moves the vector.

Relationships Require Structure

WHY FLAT CHUNKS CANNOT CONNECT FACTS

1

Vectors store chunks as isolated points

There are no edges, no foreign keys, no entity links connecting one chunk to another. Each embedding exists independently in the vector space.

2

Multi hop questions need traversable connections

Queries like 'which team members reported to the manager who approved this budget' require explicit relationship paths that flat retrieval cannot represent.

3

Knowledge graphs encode entities and edges

Nodes represent people, products, and contracts. Edges represent ownership, approval, and reporting lines. Agents traverse these connections in milliseconds.

🔗 THE CONNECTION GAP

If the answer requires joining two facts from different chunks, vectors retrieve each independently but never connect them. Graphs make that connection explicit.

Permissions Break in Flat Stores

WHY ACCESS CONTROL NEEDS STRUCTURE

Enterprise data has access controls. Vector stores treat every chunk equally, making permission enforcement an afterthought, not a guarantee.

THE VECTOR PROBLEM

Chunks from confidential HR documents sit alongside public wiki pages. Without external filtering, any query can surface restricted content to unauthorized users.

THE GRAPH SOLUTION

Knowledge graphs model ownership, org hierarchy, and classification as relationships. Permission checks become graph traversals, not post retrieval filters.

⚠ FILTERING IS NOT SECURITY

Post retrieval filtering wastes retrieval budget on chunks the user cannot see and leaks metadata about their existence. Permissions belong in the data layer.

Knowledge Graphs Fill the Gap

STRUCTURE THAT VECTORS CANNOT PROVIDE

1

Entities become nodes with typed properties

People, products, contracts, and teams are represented as nodes. Each carries structured attributes like role, status, and value that agents can filter and compare.

2

Graph queries traverse multiple hops instantly

Finding all contracts approved by a specific manager across three subsidiaries is a single graph traversal, not a search across thousands of embedded chunks.

3

Graph RAG grounds responses in explicit connections

Combining graph traversal with LLM generation produces answers grounded in verified relationships rather than semantic proximity alone.

🏆 GRAPH RAG OUTPERFORMS ON MULTI HOP

Graph RAG consistently beats vector only RAG on questions where the answer depends on connecting facts across different documents or entity boundaries.

Retrieval Methods: Side by Side

Dimension	Vector DB	Knowledge Graph	SQL Database
Best for	Fuzzy retrieval	Relationship queries	Precise lookups
Query type	Semantic similarity	Graph traversal	Structured SQL
Relationships	None (flat)	First class edges	Foreign keys
Exact answers	Approximate	Node properties	Exact values
Permissions	External filter	Graph traversal	Row level security
Agent memory	Semantic only	Episodic + semantic	Structured logs

ROUTE BY QUESTION TYPE

Production agents rarely use just one method. The strongest architectures route each query to the retrieval method that actually answers it: vectors, graphs, or SQL.

When Hybrid Search Wins

THE MINIMUM UPGRADE FROM PURE VECTORS

1

Combine dense vectors with sparse BM25 keywords

Vectors catch semantic meaning. Keywords catch exact terms, product IDs, and proper nouns that embeddings compress into indistinguishable regions of the vector space.

2

Reciprocal rank fusion merges both result sets

Results from vector and keyword retrievers are combined into a single ranked list without requiring a shared scoring function. No retraining needed.

3

Benchmarks show consistent accuracy gains

Hybrid search regularly outperforms either method alone on enterprise datasets, often improving retrieval accuracy substantially with minimal added complexity.

🛠️ HIGHEST IMPACT, LOWEST EFFORT

Adding a keyword index alongside your vector store takes hours, not weeks. It is the single highest impact improvement you can make to any RAG pipeline today.

When SQL Queries Win

PRECISE ANSWERS VECTORS CANNOT PROVIDE

1

Aggregations and filters need SQL, not similarity

How many orders shipped last month? What is the average deal size by region? These are database queries, not retrieval problems. Vectors will return related paragraphs, not numbers.

2

Text to SQL gives agents structured answers

Agents that generate validated SQL from natural language access exact values, filtered lists, and computed aggregates that no amount of chunk retrieval can produce.

3

Combine SQL results with vector context

Return the precise number from SQL and the narrative explanation from vector retrieval in a single response. The agent gets both accuracy and context.

🎯 LET THE QUESTION DECIDE

If the answer is a number, a date, or a filtered list, route to SQL. If the answer is an explanation or summary, route to vectors. Match the method to the question.

Agent Memory Needs Layers

WHY ONE INDEX IS NOT ENOUGH

1

Episodic memory needs structured storage

What happened in past conversations requires timestamps, session IDs, and user context. Vectors flatten this temporal structure into a bag of similar chunks.

2

Semantic memory fits vectors but benefits from graphs

Domain knowledge retrieves well from embeddings but gains depth when related concepts are linked through explicit graph edges rather than proximity alone.

3

Procedural memory requires rules, not similarity

How the agent should behave in specific situations needs policy engines or rule stores. Retrieving similar past behaviors is not the same as following a defined protocol.

🧠 MEMORY IS NOT A SINGLE INDEX

Human memory has distinct systems for events, knowledge, and skills. Agent memory should mirror this with different storage and retrieval strategies for each type.

Build Your Retrieval Stack

A LAYERED APPROACH TO AGENT RETRIEVAL

1

Start with Vectors

RAG over documents, wikis, and support tickets. Vectors cover the broadest range of unstructured queries with the least setup and fastest time to value.

2

Add Keyword Search

Hybrid retrieval catches IDs, product names, and technical terms that embeddings compress away. Adding BM25 alongside vectors takes hours, not weeks.

3

Add a Knowledge Graph

When users ask questions that span entities, a graph gives agents the connections vectors cannot represent. Start with your highest value entity relationships.

4

Add SQL for Precision

Aggregations, filters, and exact lookups need database queries, not similarity search. Route questions by type so every answer uses the right retrieval method.

Retrieval Mistakes to Avoid

GAPS THAT COST ACCURACY AND TRUST

🔍 Treating vector search as the only retrieval method. It handles similarity well but cannot provide structure, exact answers, or access control.

⏩ Skipping hybrid search when adding a keyword index takes hours and consistently improves accuracy over pure vector retrieval.

🔗 Ignoring relationships between entities. Multi hop questions need graphs because vectors retrieve fragments but never connect them.

🔒 Storing everything in one flat index without access controls. Enterprise data requires permission aware retrieval from day one.

📊 Using vector similarity for questions that need precise numbers. Route aggregations and exact lookups to SQL, not to embedding search.

A Retrieval Reality Check

> *Vectors find what is similar. Graphs find what is connected. SQL finds what is true. Agents that reason well use all three, routing each question to the retrieval method that actually answers it.*

— Rajesh Mane, Senior AI Engineer

Key Takeaways

📌

Vectors excel at semantic similarity

They are the right tool for fuzzy retrieval over unstructured text. But similarity is not relevance, and related is not correct.

🔗

Graphs fill the relationship gap

Knowledge graphs encode entities and connections that flat vector stores structurally cannot represent. Multi hop reasoning requires structure.

🔄

Hybrid search is the minimum upgrade

Adding keyword matching alongside vectors consistently improves accuracy. It is the highest impact, lowest effort improvement to any RAG pipeline.

🧩

Route by question type

Agents need vectors for summaries, graphs for relationships, and SQL for precise answers. The strongest retrieval stacks use all three.